HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

Logic Design

(CO3097)

Report Lab 2

SPI Protocol Module

Advisor: Ton Huynh Long

Student: Nguyen Chi Thanh 2313078

Nguyen Viet Hoang 2311066

Le Duc Tai 2312995

Mai Xuan Nhut 2312549

Ho Chi Minh City, April 12, 2026

Mục lục

1 Interface

1.1 Master Module

The figure below shows the top-level block diagram of the Master module, including its internal submodules and the connections between them.

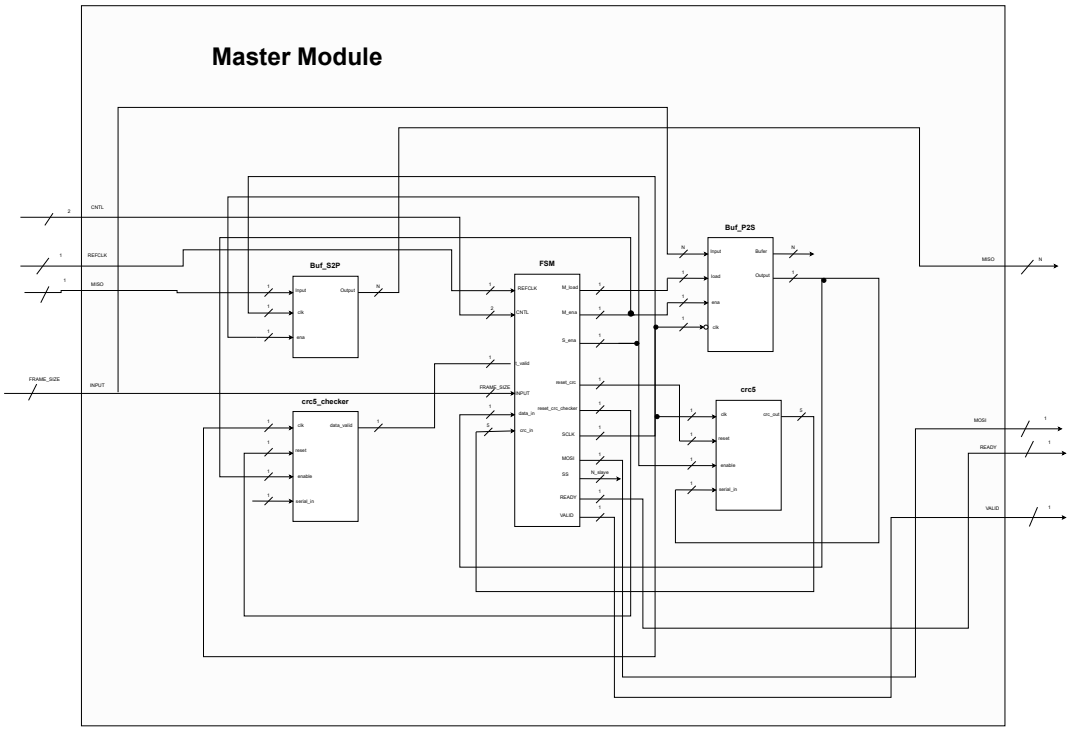

Image 1.1: Master top-level block diagram

The main submodules are Buf_P2S, Buf_S2P, crc5, and crc5_checker.

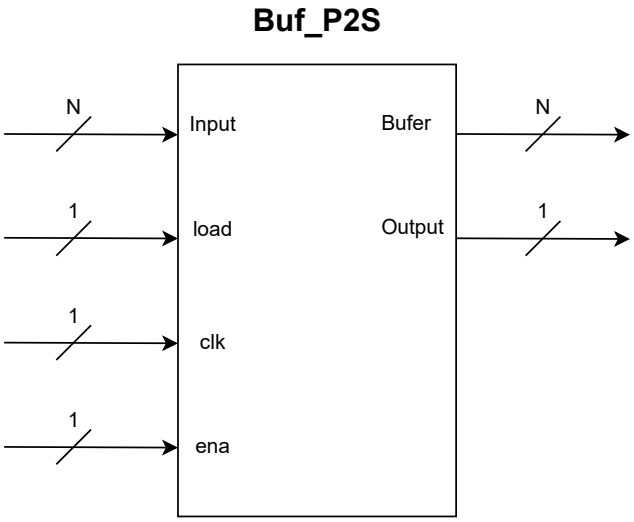

Image 1.2: Buf_P2S

Buf_P2S

Parameter: N = 8 (default).

Inputs:

– Input[N-1:0]: parallel input data (N bits).

– load: signal used to load data into the buffer.

– clk: clock signal for data shifting.

– ena: shift enable signal.

Outputs:

– Output: serial output data (1 bit).

– Buffer[N-1:0]: buffer content in parallel form.

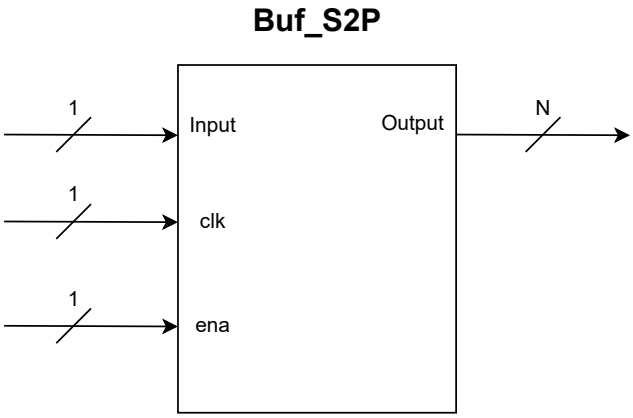

Image 1.3: Buf_S2P

Buf_S2P

Parameter: N = 8 (default).

Inputs:

– Input: serial input data (1 bit).

– clk: clock signal.

– ena: shift enable signal.

Outputs:

– Output[N-1:0]: parallel data after receiving all N bits.

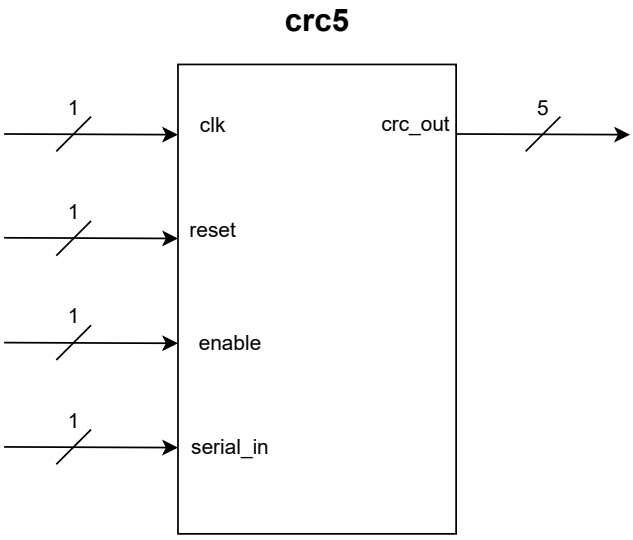

Image 1.4: crc5

crc5

Inputs:

– clk: clock signal.

– reset: resets the CRC value to 0.

– enable: enables CRC update.

– serial_in: serial input data (1 bit).

Outputs:

– crc_out[4:0]: current CRC-5 value.

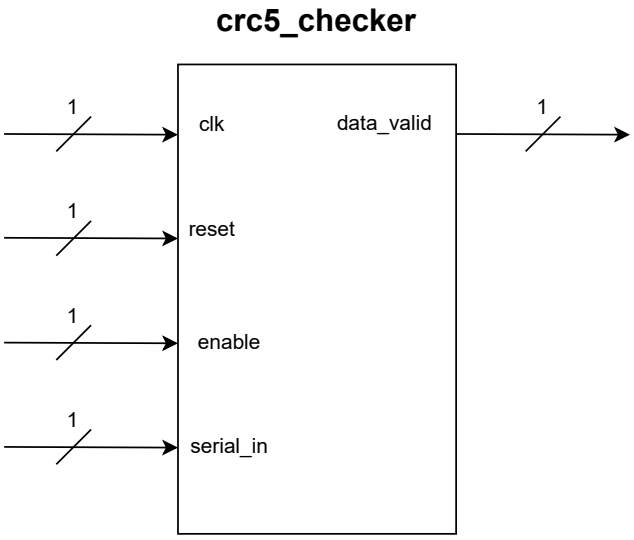

Image 1.5: crc5_checker

crc5_checker

Inputs:

– clk: clock signal.

– reset: resets the checker logic.

– enable: enables CRC checking on the received serial stream.

– serial_in: serial input data (1 bit).

Outputs:

– data_valid: indicates that the received data is valid when the CRC is correct.

– check_crc_out[4:0]: current checked CRC value, if observation is required.

1.2 Slave Module

The main block is SPI_Slave_CRC, which internally uses the submodules Parallel_to_serial, serial_to_parallel, crc5, and crc5_checker.

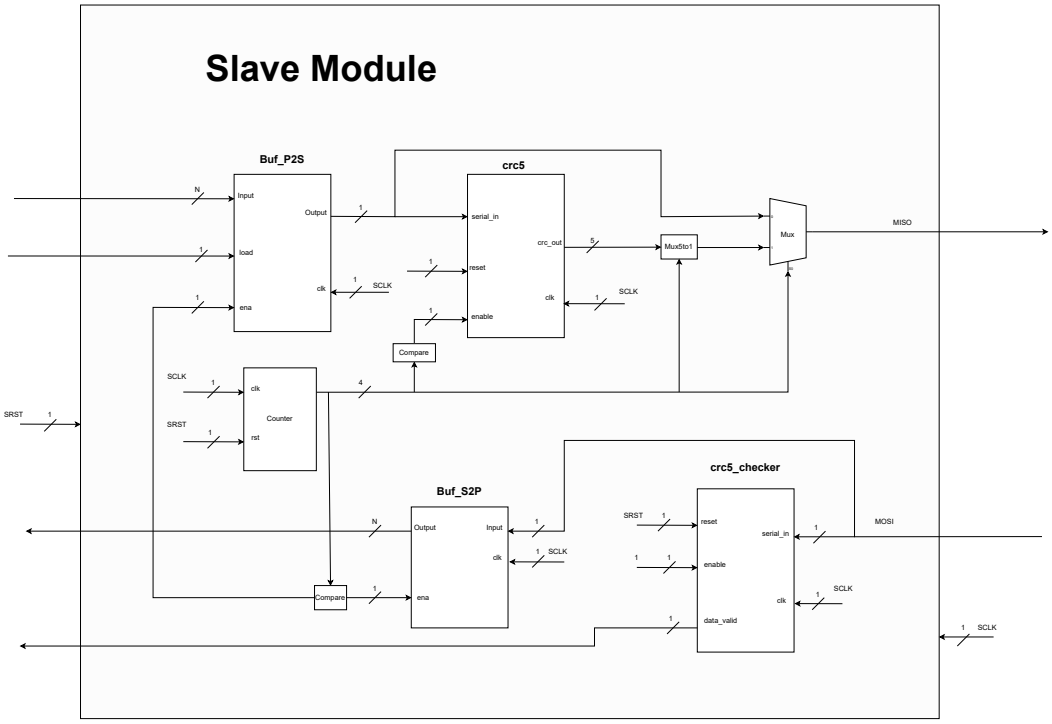

Image 1.6: Slave top-level block diagram

SPI_Slave_CRC

Parameters: n=8, crc_len=5, total_len=n+crc_len, cnum=4, CPOL=0, CPHA=0.

Inputs:

– n_rst: active-low reset.

– data_in[n-1:0]: parallel data provided by the Slave for MISO transmission.

– load: loads data into the P2S block.

– sclk: SPI clock from the Master.

– MOSI: serial data from the Master.

– n_cs: active-low chip select.

Outputs:

– data_out[n-1:0]: received parallel data after collecting all N bits.

– MISO: serial data returned to the Master, including payload and CRC bits.

– rx_data_valid: indicates that the received data is valid when the CRC is correct.

– rx_done: indicates that the full frame, including data and CRC, has been received.

– crc_out[crc_len-1:0]: CRC generated for the transmitted data.

– ready: indicates that the Slave is ready for transmission or reception.

Parallel_to_serial

Function: converts the parallel input data_in[n-1:0] into the serial stream data_out when load and ena are asserted.

serial_to_parallel

Function: captures serial input data data_in and reconstructs it into parallel output data_out[n-1:0] when ena is active.

2 Internal Implementation

This section elaborates on the internal hardware architecture, the hierarchical module decomposition, the algorithmic Finite State Machine (FSM) design, and the synchronous data shifting mechanisms of the SPI communication system integrated with a CRC5 error-checking block.

2.1 System Hierarchy and Structural Modeling

The hardware system is designed using a structural modeling approach, partitioning complex data link operations into fundamental, reusable functional blocks. The hierarchical composition of the design is as follows:

- **SPI_Master_CRC** (Master Controller with Error Checking)
  - Buf_P2S: Parallel-to-Serial buffer for transmission mapping.
  - Buf_S2P: Serial-to-Parallel buffer for reception mapping.
  - crc5: Computes the 5-bit Cyclic Redundancy Check (CRC) checksum for the outgoing MOSI data stream.
  - crc5_checker: Validates the integrity of the incoming data on the MISO line against transmission errors.
- **SPI_Slave_CRC** (Peripheral Module with Error Checking)
  - serial_to_parallel: Deserializes incoming MOSI bitstreams into parallel byte structures.
  - Parallel_to_serial: Serializes the outgoing payload onto the MISO interface.
  - crc5: Evaluates the CRC5 checksum for the Slave's transmitted data.
  - crc5_checker: Decodes and verifies the payload received from the Master.

2.2 SPI Master Module Implementation

2.2.1 Data Shifting Mechanism

The Buf_P2S module is responsible for capturing parallel data from the INPUT bus upon the assertion of the load signal. The shifting process is synchronized with the transmission clock edges. During each active shift cycle (ena == 1), the least significant bit is pushed directly to the Output pin. Simultaneously, the internal array is right-shifted:

$$\text{buffer[i-1]} \leftarrow \text{buffer[i]} \quad (\text{for } i = N-1 \text{ down to } 1) \tag{2.1}$$

The most significant bit, buffer[N-1], is subsequently flushed to 1'b0 to clear the register. Conversely, the Buf_S2P logic continuously samples the MISO line, shifting the most recent bit into buffer[N-1] and propagating older bits downwards.

2.2.2 Finite State Machine (FSM) Architecture

The core control unit of the Master operates as an FSM synchronized to the rising edge of the system reference clock, REFCLK. The machine traverses four primary operational states:

1. **IDLE (2'b00):** The resting state. The READY and VALID flags are maintained HIGH. The system actively polls the CNTL bus from the host CPU to determine the next state transition, while holding the CRC computing engines in a reset state.

2. **LOAD_DT (2'b01):** The data loading state. The internal M_load control signal is asserted to latch the transmission byte from the INPUT bus into the shift register.

3. **LOAD_CS (2'b10):** The peripheral configuration state. The value stored in the input buffer is decoded to assert the corresponding active-LOW Slave Select (SS) line using the following logic:

```
SS <= (buf_input < Number_Slave) ? {Number_Slave{1'b1}} & ~(1 << buf_input)
                                 : {Number_Slave{1'b1}};
```

4. **TRANSFER (2'b11):** The active communication state. The READY flag is pulled LOW, isolating the system from external write attempts, and the internal count variable begins tracking the shifted bits.

2.2.3 Frame Multiplexing and CRC Integration

During the TRANSFER state, the communication frame is extended to 13 clock cycles (8 bits payload + 5 bits CRC), corresponding to the count variable incrementing from 0 to Frame_Size + 4.

– **Data Phase (count $<$ Frame_Size):** The MOSI line is driven directly by the output of the Buf_P2S register. The crc5 module actively calculates the checksum of the bitstream being transmitted.

– **CRC Phase (count $\geq$ Frame_Size):** The Master shifts its multiplexer to directly output the resultant bits from the internal CRC calculation array (crc_out):

$$\text{MOSI} \leftarrow \text{crc\_out}[\text{Frame\_Size} + 4 - \text{count}] \tag{2.2}$$

2.3 SPI Slave Module Implementation

Unlike the Master, which relies on a complex FSM structure, the SPI_Slave_CRC module is architecturally optimized around a counter-driven execution model.

2.3.1 Clock Domain Synchronization

To natively support varying Clock Polarity (CPOL) and Clock Phase (CPHA) configurations, the module incorporates XOR-based combinational logic to synthesize the internal execution clock (inter_clk):

$$\text{inter\_clk} = \text{sclk} \oplus \text{CLK\_CTRL} \tag{2.3}$$

where $\text{CLK\_CTRL} = \text{CPOL} \oplus \text{CPAH}$. All serial capture and shift-out operations are executed strictly on the rising edge of inter_clk, ensuring robust data boundary alignments.

2.3.2 Counter-Driven Logic and Tri-State Output

The module maintains an internal 4-bit counter. This counter evaluates incoming edges sequentially from 0 to 13 as long as the chip select (n_cs) is asserted LOW. Data shifting arrays are explicitly enabled only during the first 8 payload cycles.

Furthermore, to accommodate multi-slave bus topologies, the MISO output is guarded by a tri-state multiplexer:

$$\text{MISO} = \begin{cases} \text{1'bz} & \text{if n\_cs} = 1'b1 \\ \text{tx\_miso\_data} & \text{if counter} < \textit{Frame\_Size} \\ \text{tx\_crc\_out}[\ldots] & \text{if } \textit{Frame\_Size} \leq \text{counter} < 13 \end{cases} \tag{2.4}$$

This effectively floats the bus when the Slave is dormant, avoiding electrical conflicts.

2.4 Mathematical Operator: CRC5 Implementation

The error detection mechanism is realized via a Linear Feedback Shift Register (LFSR). The combinational logic for the polynomial feedback operates synchronously:

```
assign feedback = serial_in ^ crc_out[4];
always @(posedge clk or posedge reset) begin
    if (reset) begin
        crc_out <= 5'b00000;
    end else if (enable) begin
```

```
        crc_out[0] <= feedback;
        crc_out[1] <= crc_out[0];
        crc_out[2] <= crc_out[1] ^ feedback;
        crc_out[3] <= crc_out[2];
        crc_out[4] <= crc_out[3];
    end
end
```

The verifier module, crc5_checker, streams the received serial bits through an identical LFSR framework. If the payload is uncorrupted, the residual value in the shift registers will evaluate to exactly zero at the end of the 13th cycle. This state is detected via a NOR reduction over the register array, immediately asserting the data_valid flag to certify the integrity of the transaction.

3 Simulation

This section presents a detailed overview of the verification test cases designed based on the validation checklist. These simulation runs aim to confirm the operational correctness and robustness of the complete SPI communication system integrated with the autonomous CRC5 error detection mechanism.

3.1 Slave Module Verification (Slave Module Check)

The Slave module verification is derived from the validation checklist and the scenarios implemented in tb_SPI_Slave_CRC.v. This part checks the key Slave-side functions, including correct 13-bit frame reception from the Master, CRC error detection, data transmission from the Slave back to the Master through MISO, the expected ready/valid behavior, and the active-low chip-select condition controlled by n_cs.

– Receiving a valid 13-bit frame from the Master, consisting of 8 data bits and a correct 5-bit CRC.
– Receiving an invalid 13-bit frame with an incorrect CRC.
– Transmitting data from the Slave back to the Master and verifying the transmitted data and CRC.
– The Slave's behavior when n_cs is not asserted (active high) while the Master is transmitting.

Slave_TH1 – Valid data reception from Master In the first scenario, the Master sends a valid 13-bit frame to the Slave through MOSI. The transmitted payload is 0x3C and the accompanying CRC field is 10010, which matches the expected CRC5 result for this frame. The waveform confirms that the Slave reconstructs the 8-bit output correctly after the full transaction, satisfying the checklist item slave_rx_correct_crc. At the end of the frame, Slave_valid remains asserted, indicating that the received data passes CRC checking. The Slave_ready signal also follows the expected behavior: it is active when the Slave is idle and becomes inactive while the frame is being received.

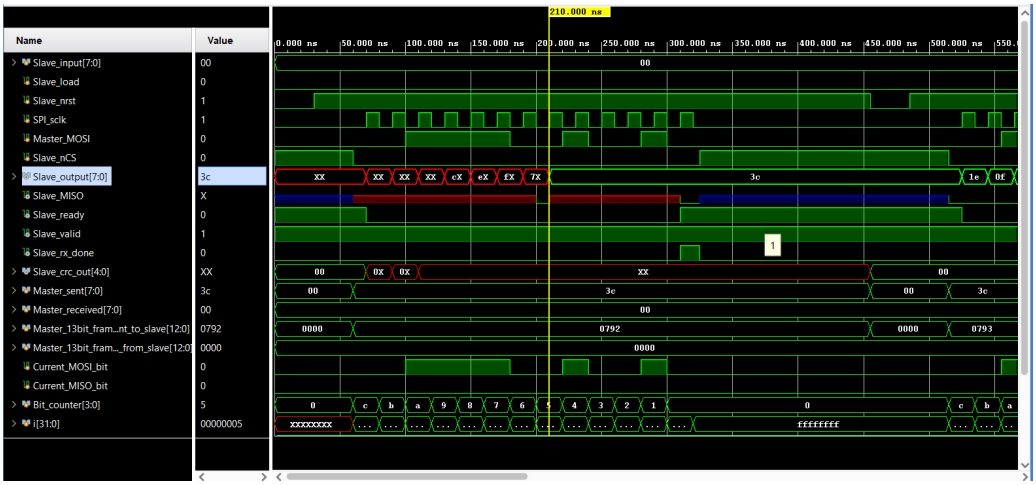

Image 3.1: Slave_TH1: The Master sends a valid 13-bit frame with data 0x3C and correct CRC, and the Slave accepts the frame successfully.

Slave_TH2 – Invalid data reception from Master The second scenario verifies the CRC error-detection path on the Slave side. The Master again sends a 13-bit frame with payload 0x3C, but this time the CRC field is intentionally corrupted to 10011. The Slave still samples all incoming bits on MOSI and reconstructs the frame timing correctly, but at the final validation cycle Slave_valid deasserts to 0. This behavior matches the checklist item slave_rx_wrong_crc and confirms that the Slave rejects frames whose 5-bit CRC field is incorrect.

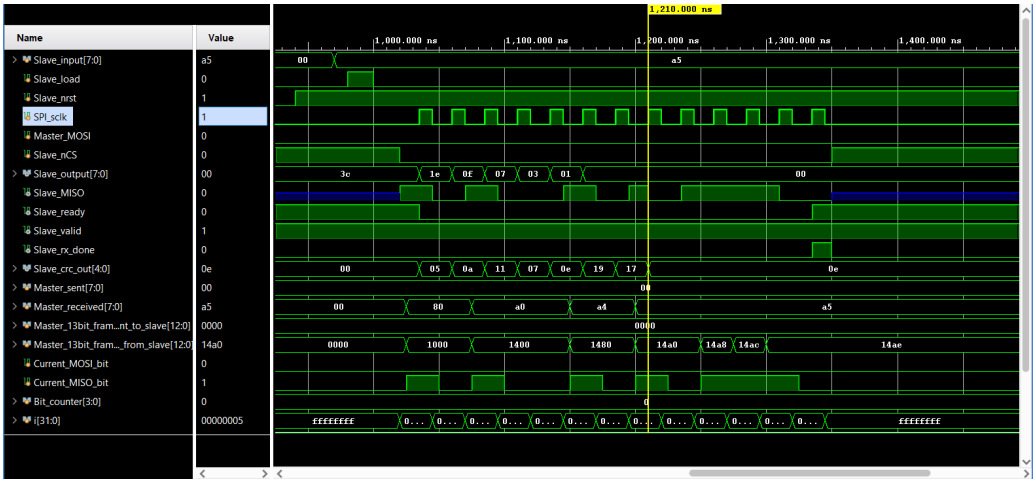

Image 3.2: Slave_TH2: The Master sends a 13-bit frame with an incorrect CRC, and the Slave detects the error by forcing Slave_valid low.

Slave_TH3 – Data transmission from Slave to Master The third scenario verifies the transmit behavior of the Slave, corresponding to the checklist item slave_tx_data_to_master. In this case,

the Slave loads 0xA5 into its internal transmit buffer and then shifts the data back to the Master on MISO while n_cs remains active low. The waveform shows that the Master-side receiver observes the correct 8-bit payload, and the full 13-bit transmitted frame contains the expected CRC field generated by the Slave. This confirms that the Slave correctly serializes its buffered data and appends the CRC bits during transmission.

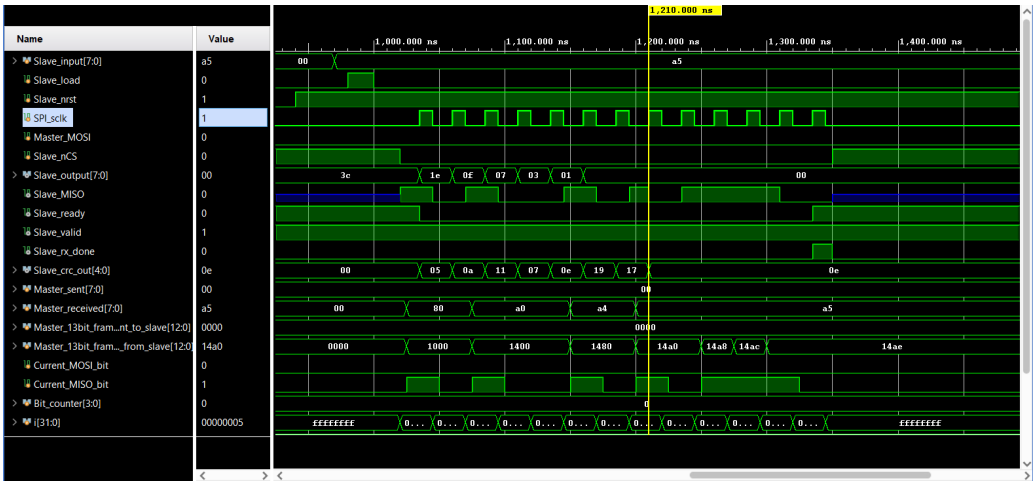

Image 3.3: Slave_TH3: The Slave transmits data 0xA5 to the Master through MISO together with the correct CRC bits.

Slave_TH4 – Slave not selected while Master transmits The fourth scenario verifies the active-low chip-select behavior and complements the checklist items slave_ready_behavior and slave_chip_select_behavior. Here, the Master still drives a 13-bit frame on MOSI, but the Slave input n_cs is kept high for the entire transaction. Because the Slave is not selected, it does not shift in the incoming bits, does not accept the frame, and remains inactive from the communication point of view. The waveform confirms that the Slave only participates in reception or transmission when n_cs is asserted low, while deselection keeps the internal receive operation disabled and preserves the expected idle behavior.

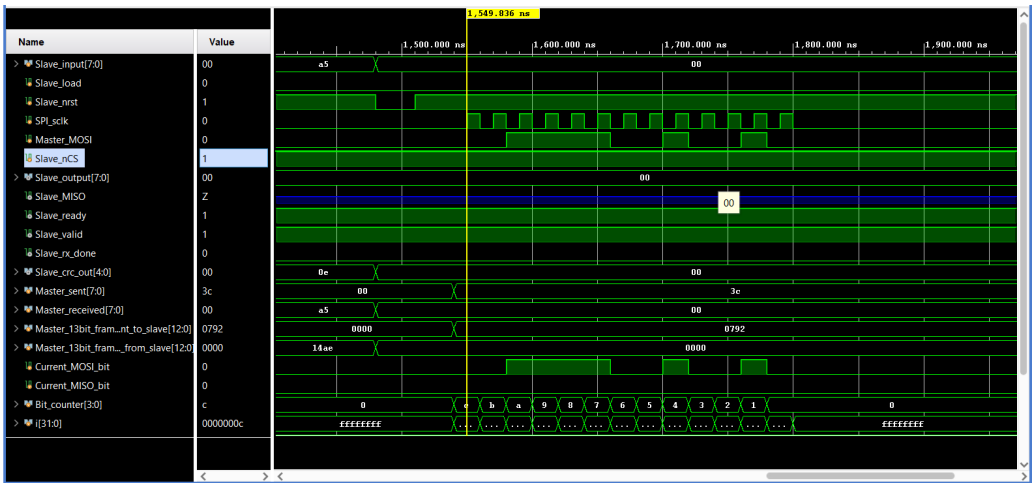

Image 3.4: Slave_TH4: The Master transmits while n_cs remains high, so the Slave stays deselected and does not capture the incoming frame.

3.2 Master Module Verification (Master Module Check)

The Master module verification is derived from the validation checklist and the implemented scenarios in tb_SPI_Master_CRC.v. This part checks the key Master-side functions, including correct 13-bit frame reception from a virtual Slave, CRC error detection, active-low Slave selection, control sequencing through CNTL, transfer-clock generation on Master_SCLK, and the expected READY/VALID behavior before, during, and after transmission.

– Receiving a valid 13-bit frame from a virtual Slave, including 8 data bits and a correct 5-bit CRC.

– Receiving a 13-bit frame with an incorrect CRC and verifying that the Master detects the error correctly.

– Selecting the target Slave through active-low Master_SS, including the invalid-selection case where no Slave is chosen.

– Loading transmit data into the Master and sending the data together with CRC bits through MOSI.

– Verifying the control flow of CNTL, the generation of Master_SCLK, and the expected READY/VALID behavior across the transfer.

Master_TH1 – Valid data reception from Slave In the first scenario, the Master verifies a correct receive transaction from the Slave side. The testbench selects Slave index 0 by applying INPUT = 0x00 with CNTL = 2, then a virtual Slave transmits a 13-bit frame on MISO with data 0x3C

and CRC 10010. This is the same valid 13-bit pattern used in the first Slave-side verification case, allowing direct comparison between the receive behavior of both modules. During CNTL = 3, the Master samples the incoming frame using Master_SCLK; at the final checking cycle, Master_output matches the expected 8-bit payload and Master_valid remains high, confirming successful CRC verification. The waveform also shows that Master_ready is asserted while the Master is idle or configuring the transaction and deasserted during the active transfer interval, which is consistent with the design and the checklist expectations.

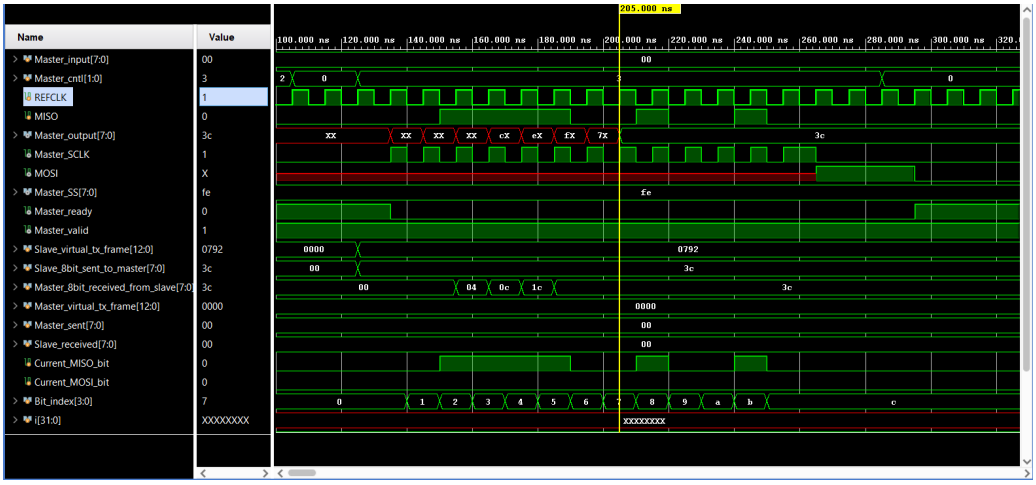

Image 3.5: Master_TH1: The Master receives a valid 13-bit frame from the virtual Slave (data = 0x3C, correct CRC), and Master_valid stays high at the final check cycle.

Master_TH2 – Invalid CRC reception and invalid Slave selection The second scenario checks the error-detection path of the Master. The virtual Slave sends another 13-bit frame to the Master, but the CRC field is intentionally incorrect. In this testbench, the transmitted data is 0xCC and the CRC pattern is corrupted, so the CRC5 checker inside the Master rejects the frame. As a result, at the last checking clock of the transfer, Master_valid falls to 0, indicating CRC failure exactly as required by the checklist item master_rx_wrong_crc. At the same time, the Slave-select logic is also verified using an invalid selection input: when CNTL = 2 and INPUT = 0x0F, the selected index is outside the valid range for 8 Slaves, so Master_SS remains 0xFF. This means no Slave is selected, which correctly matches the active-low Slave-select specification.

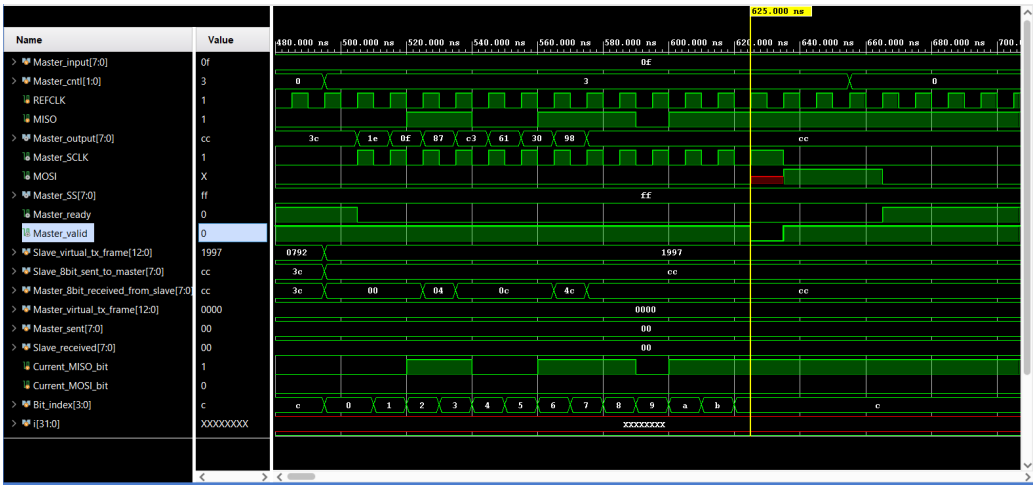

Image 3.6: Master_TH2: The Master detects a CRC error from the virtual Slave, Master_valid drops low at the final check cycle, and invalid selection input 0x0F keeps Master_SS = 0xFF.

Master_TH3 – Master transmits data to Slave 3 The third scenario verifies the transmit path from the Master to a selected Slave. First, the testbench loads 0x33 into the Master transmit buffer using CNTL = 1. Next, Slave index 3 is selected with CNTL = 2 and INPUT = 0x03. Based on the active-low select logic implemented in SPI_Master_CRC, this produces Master_SS = 11110111, meaning only Slave 3 is enabled. The transfer then starts when CNTL = 3, and the outgoing 13-bit frame on MOSI is captured by the testbench. The waveform confirms that the Master transmits the 8-bit payload 0x33 together with its CRC bits in the expected order, while Master_SCLK is generated only during the transfer phase. Therefore, this case validates the checklist items related to master_tx_data_to_slave, master_slave_select, master_cntl_state_control, and master_sclk_generation.

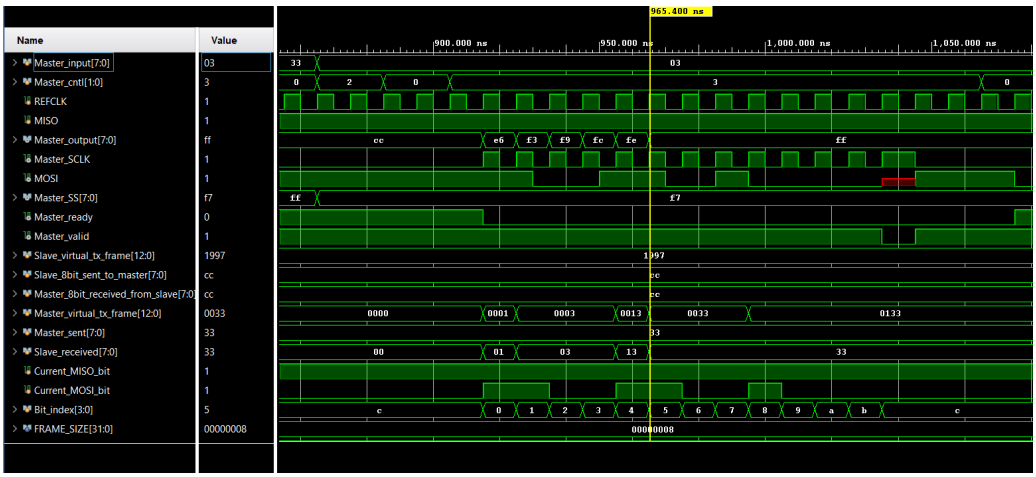

Image 3.7: Master_TH3: The Master loads data 0x33, selects Slave 3 so that Master_SS = 11110111, and transmits the 13-bit MOSI frame correctly.

3.3 Loop Back Verification (Loop Back Check)

The loopback verification focuses on continuous data circulation between the Master and Slave after both modules are connected together. Based on tb_SPI_Circular_Loopback.v, this test checks the loopback_transfer behavior in the checklist by confirming that the Master can first send data to the Slave, then the Slave stores the received value, reloads it into its transmit buffer, and returns the same value back to the Master in the next transaction. The waveform also confirms that the handshake signals operate according to the specification: m_ready is asserted in the idle or setup phases, deasserted during transfer, and m_valid indicates successful frame completion after CRC checking.

– Sending data from the Master to the Slave in the first transaction of the loopback sequence.
– Storing the received value inside the Slave and reloading it into the Slave transmit path.
– Returning the same data from the Slave back to the Master in the next transaction.
– Verifying continuous circular communication behavior between both modules.
– Checking that SS, SCLK, MOSI, MISO, ready, and valid operate consistently throughout the loopback process.

LoopBack_TH1 – Circular loopback transfer In this testbench, the initial payload loaded into the Master is 0x99. During the first loop, the Master loads the data, selects Slave 0, and transmits the frame to the Slave through MOSI. The Slave receives and stores the transmitted value correctly. In the second loop, the Slave loads its output data back into the transmit path and sends the same value 0x99 to the Master through MISO. At the same time, the Master also reloads its input path to continue the communication cycle. The final waveform confirms that the data returning to the Master matches the original transmitted value, proving that the circular ping-pong loopback mechanism works correctly. This case also demonstrates correct synchronization between SS, SCLK, MOSI, and MISO, since data movement only occurs while the Slave is selected and the transfer clock is active.

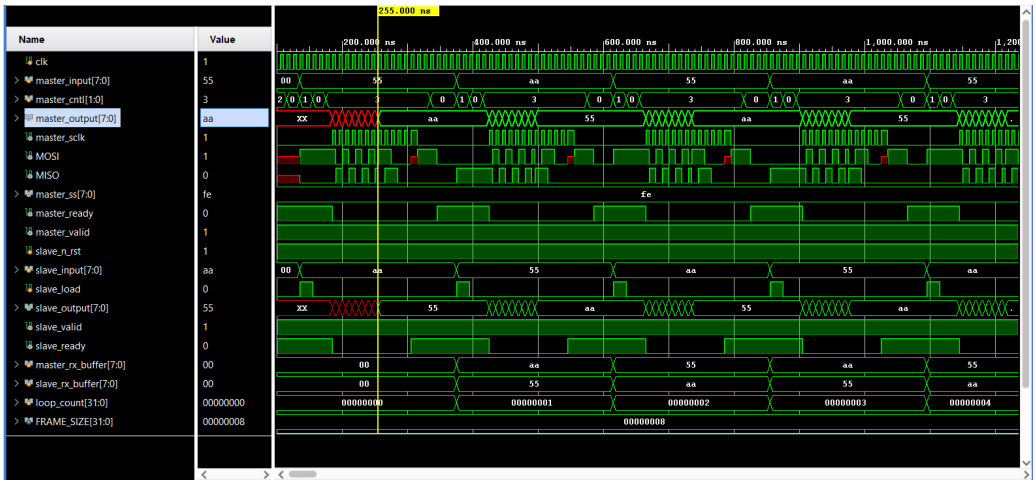

Image 3.8: LoopBack_TH1: The Master sends 0x99 to the Slave, the Slave stores and returns the same value, and the circular loopback transfer is verified successfully.

3.4 Integrated System Verification (Full SPI Communication Check)

The integrated system verification uses the complete Master-Slave connection in tb_Com_Full_Module.v to validate end-to-end SPI behavior. This subsection covers the main checklist items for the full communication system: master_to_slave_transfer, slave_to_master_transfer, full_duplex_transfer, crc_error_detection_integrated, ss_sclk_sync_integrated, and idle_to_transfer_recovery. The simulation is organized into three practical communication phases that together verify the complete integrated behavior of the design.

– Transmitting data from the Master to the Slave and verifying correct reception on the Slave side.

– Transmitting data from the Slave to the Master and verifying correct reception on the Master side.

– Verifying simultaneous full-duplex communication where both modules exchange data within the same frame.

– Checking CRC validation behavior in the integrated communication path.

– Confirming synchronization between SS, SCLK, MOSI, and MISO during active transfer.

– Verifying that the system transitions cleanly from idle to transfer and returns to idle without stale data or control errors.

Full_Module_TH1 – Master-to-Slave transfer In the first integrated scenario, the Master sends data to the Slave through the normal SPI path. The testbench loads 0xD4 into the Master transmit input, selects Slave 0 by driving INPUT = 0x00 during the select phase, and then starts the transfer.

The waveform confirms that the data placed on MOSI is correctly captured by the Slave, and the received Slave output matches the transmitted Master payload. At the end of the frame, the Slave-side CRC validation signal indicates successful reception, confirming the integrated master_to_slave_ transfer path. This phase also shows that the transaction occurs only when SS[0] is active low and SCLK is generated, which verifies the expected SS-SCLK synchronization behavior.

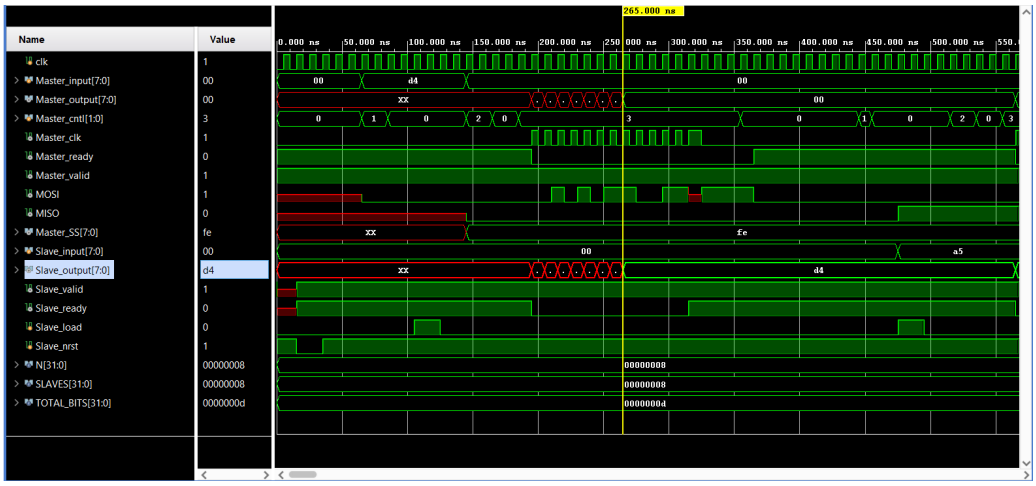

Image 3.9: Full_Module_TH1: The Master transmits data 0xD4 to Slave 0, and the Slave receives the frame correctly with successful CRC validation.

Full_Module_TH2 – Slave-to-Master transfer The second integrated scenario verifies the reverse communication path from the Slave back to the Master. Here, the Slave loads 0xA5 into its transmit buffer, while the Master keeps the Slave selected and performs a receive transaction. The waveform shows that the Slave drives the data onto MISO, and the Master reconstructs the expected 8-bit output correctly after one full frame. This confirms the slave_to_master_transfer behavior and demonstrates that the Master-side receive and CRC-checking logic operate correctly in the integrated connection. The ready/valid behavior also follows the expected protocol: the Master leaves the idle state, completes the transfer, and then returns to ready state once CNTL is deasserted back to idle.

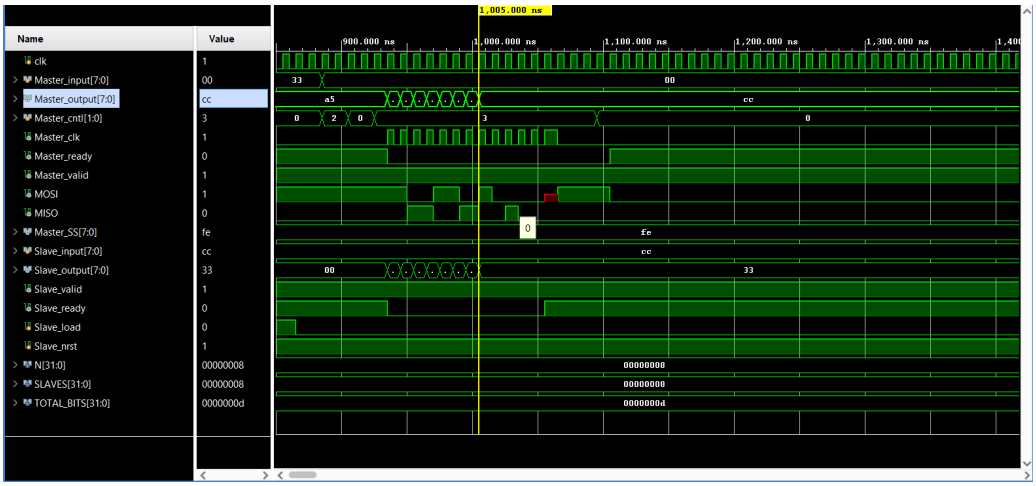

Image 3.10: Full_Module_TH2: The Slave transmits data 0xA5 to the Master through MISO, and the Master receives the correct output after frame completion.

Full_Module_TH3 – Full-duplex integrated transfer The final integrated scenario verifies simultaneous bidirectional communication. The Master loads 0x33 for transmission while the Slave loads 0xCC into its own transmit buffer. When the transfer begins, the Master sends data to the Slave on MOSI while the Slave simultaneously sends data back to the Master on MISO. The resulting waveform confirms that both modules receive the opposite-side payload correctly after a single complete transaction, which verifies the full_duplex_transfer requirement. In addition, this phase demonstrates correct transition from idle to transfer and back to idle without stale data or counter errors, while READY and VALID remain consistent with the protocol specification. Together with the previous phases, this confirms that the integrated system satisfies the intended functional behavior of the complete SPI communication architecture.

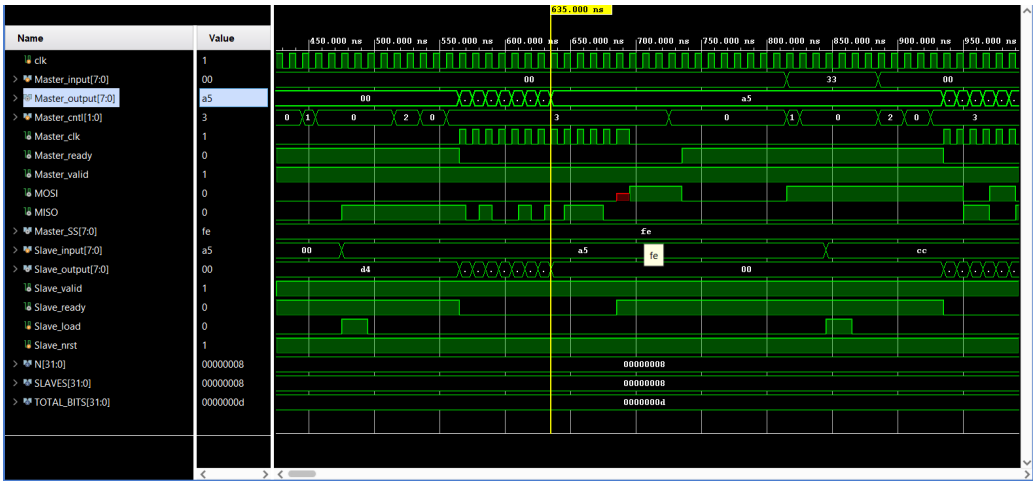

Image 3.11: Full_Module_TH3: The Master sends 0x33 while the Slave simultaneously sends 0xCC, verifying correct full-duplex SPI communication.

4 Synthesis and Results Summary

4.1 Cadence Synthesis Flow

The SPI_Top design was synthesized using Cadence Genus. The top-level architecture connects one master module and two slave modules into a single system-level block, together with the supporting serial/parallel buffers and CRC logic.

After synthesis, the key generated files are:

– Netlist: outputs??/$DESIGN_m.v (for example, bound_flasher_m.v)

– Area report: reports??/final_area.rpt

– QoR report: reports??/final_qor.rpt

– Timing report: reports??/final_time.rpt

4.2 Netlist Output

The synthesized netlist describes the structural implementation of SPI_Top after logic optimization. It includes the master block, slave blocks, CRC logic, and the required multiplexing and control paths. This file is the main reference for the final gate-level structure.

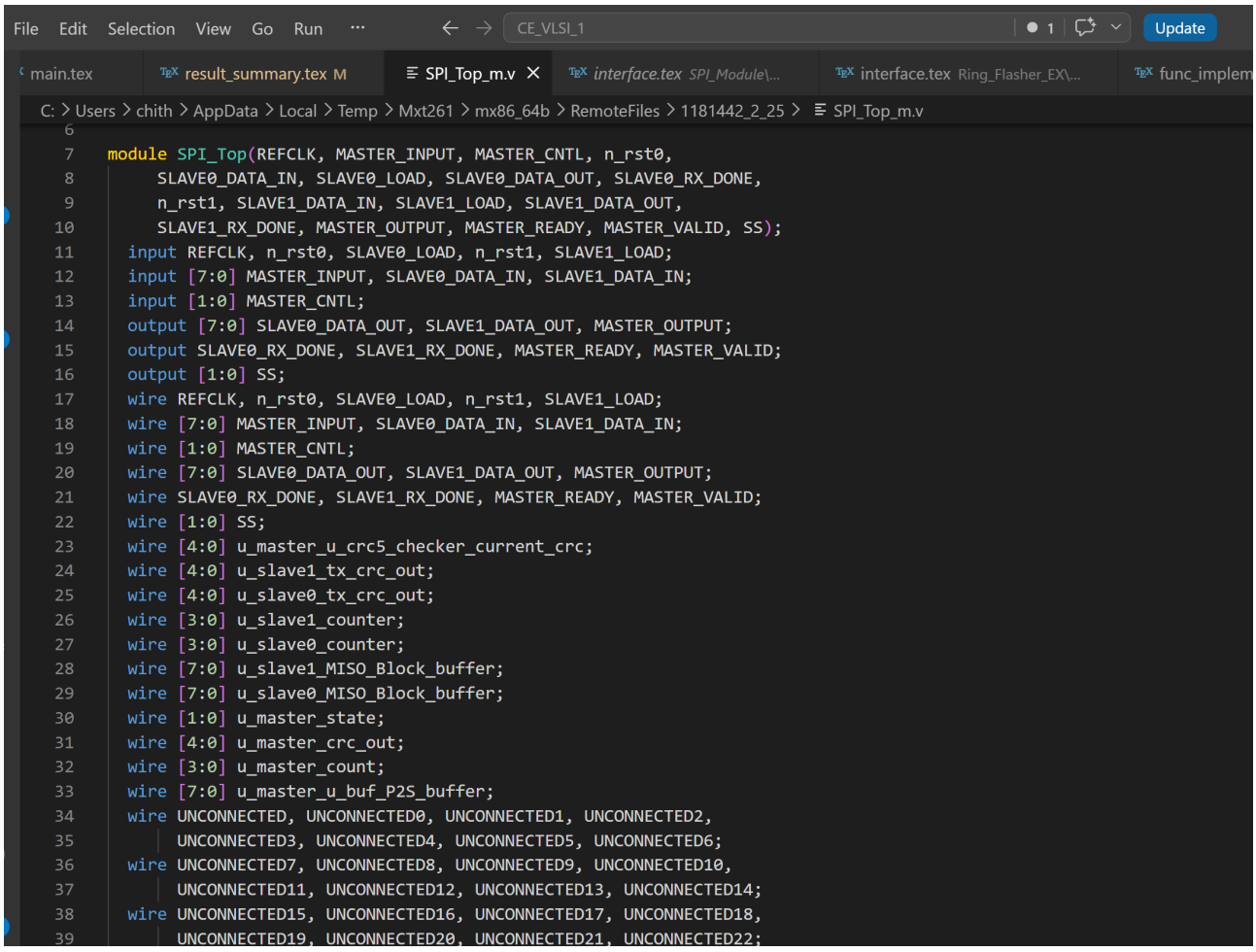

Image 4.1: Generated netlist file after synthesis

4.3 Area Report

The area report shows the compactness of the synthesized design. According to final_area.rpt, the top module occupies 246 cells with a total area of 1231.254 μm^2.

Metric	Value	Unit
Cell Count	246	cells
Cell Area	933.660	μm^2
Net Area	297.594	μm^2
Total Area	1231.254	μm^2

Bảng 4.1: Area summary extracted from final_area.rpt

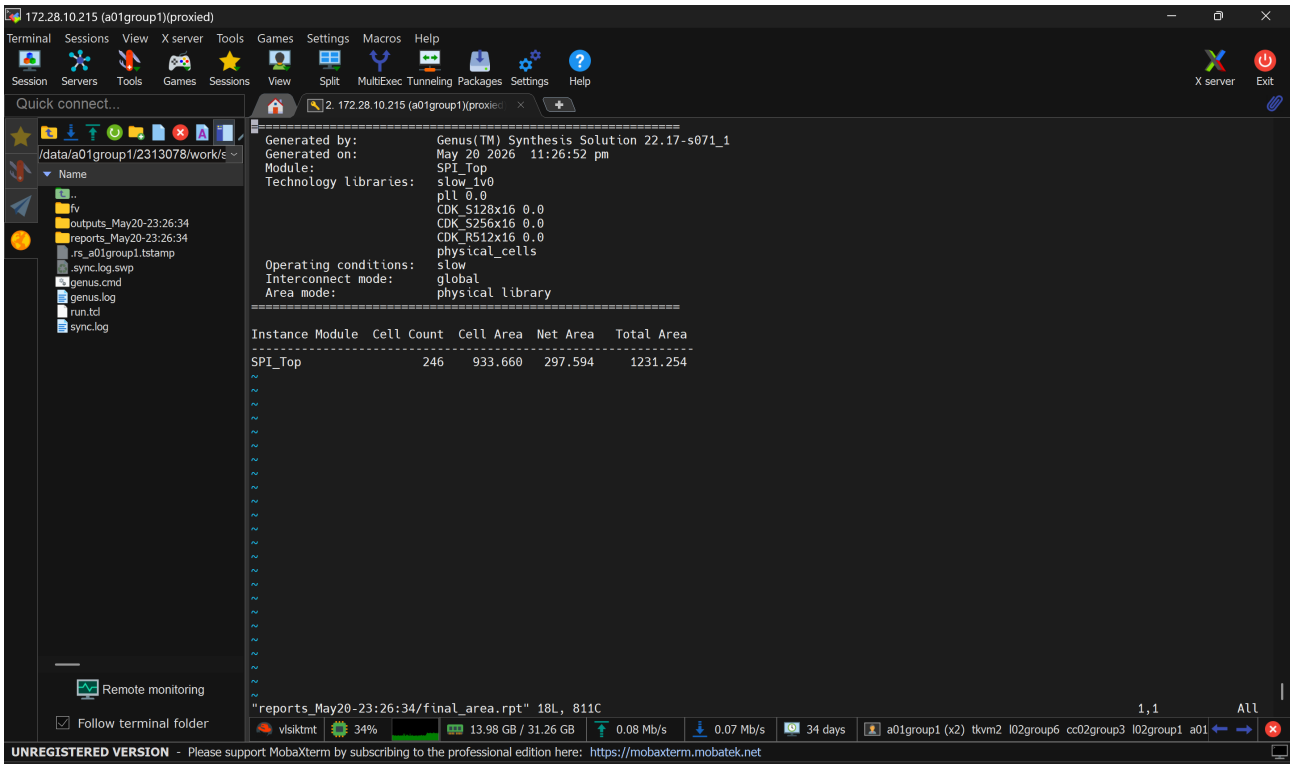

Image 4.2: Cadence area report for SPI_Top

Instance breakdown from the QoR summary:

– Leaf Instance Count: 246

– Sequential Instance Count: 84

– Combinational Instance Count: 162

– Hierarchical Instance Count: 0

4.4 QoR Report

The QoR report summarizes the overall synthesis quality. For this design, the REFCLK group shows no violating paths, and the critical slack is positive. The design therefore meets timing at the synthesized operating point.

Group	Slack (ns)	Violating Paths
REFCLK	3384.2	0
default	No paths	0

Bảng 4.2: QoR summary extracted from final_qor.rpt

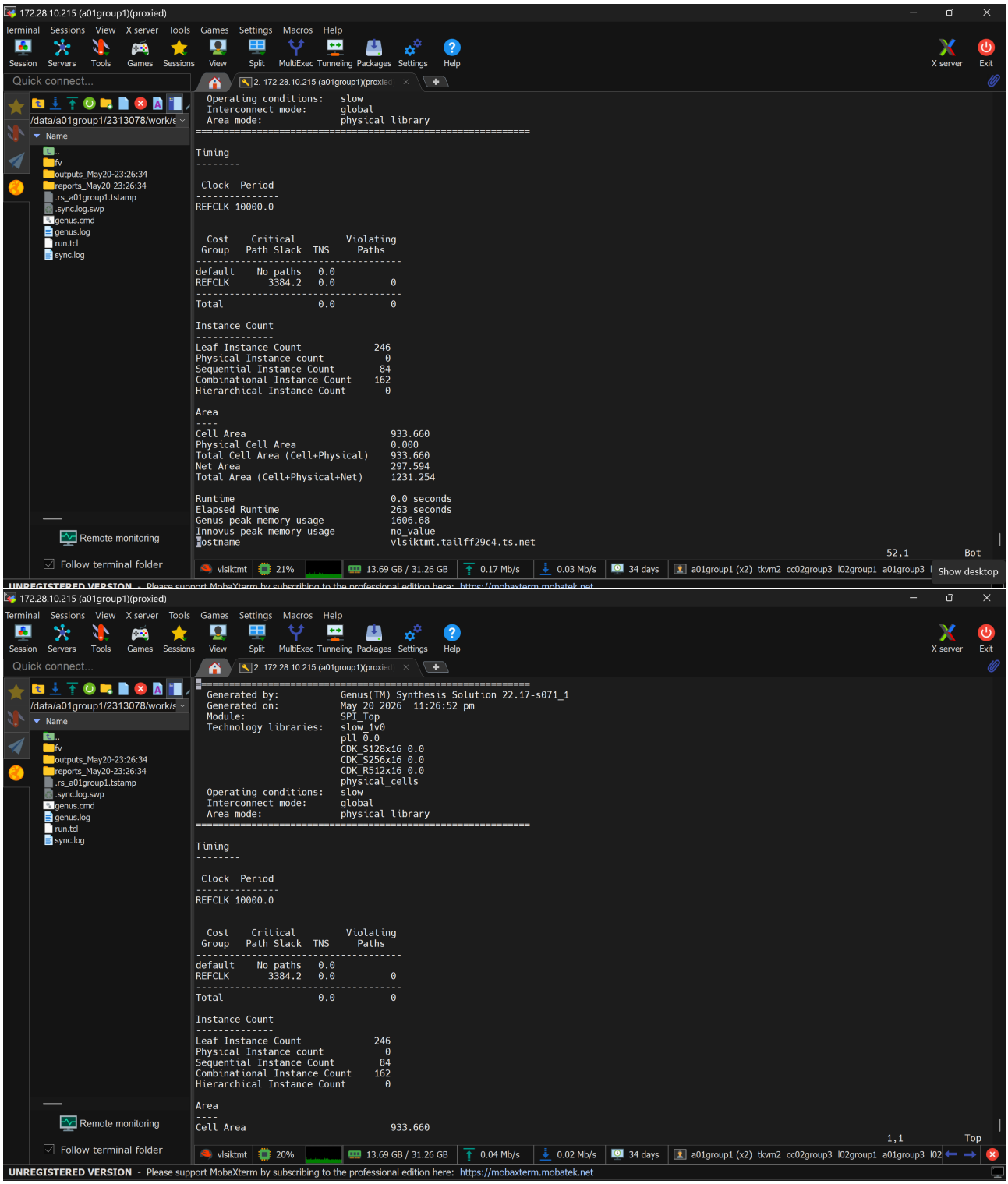

Image 4.3: Cadence QoR report for SPI_Top

4.5 Timing Report

The timing report in final_time.rpt confirms that the major internal paths meet setup timing. The most critical paths are related to the master buffer registers, CRC logic, and the slave transmit paths. All reported setup checks are marked MET.

Timing Item	Value	Status
Worst setup slack	3384 ps	MET
Worst observed late external delay	7210 ps	MET
Total violating paths	0	PASS

Bảng 4.3: Timing summary extracted from final_time.rpt

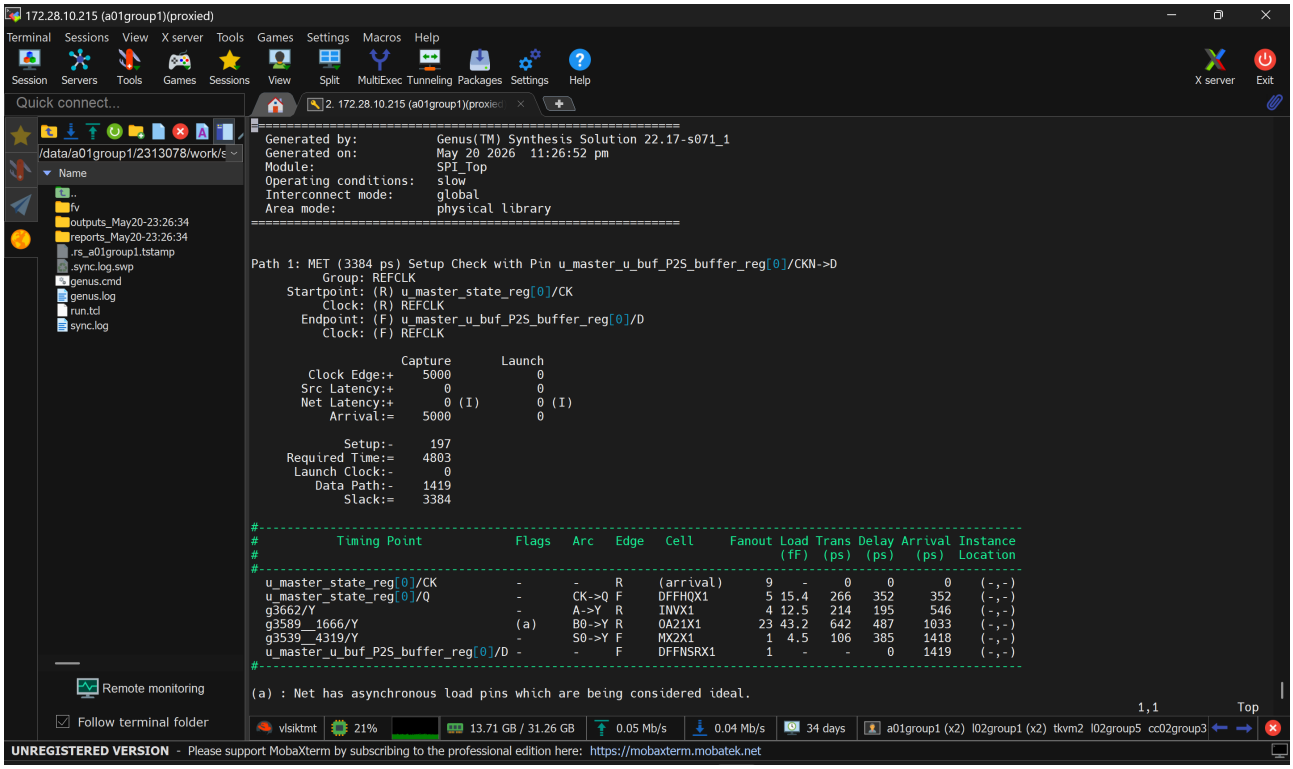

Image 4.4: Cadence timing report for SPI_Top

4.6 Conclusion

The Cadence synthesis results confirm that the SPI_Top design is structurally compact and timing-clean. The generated netlist, area report, QoR report, and timing report all indicate that the

design is ready for the next verification or implementation stage.

– The synthesized netlist contains the expected master-slave system structure.
– The area result remains compact at 1231.254 μm^2.
– The QoR report shows positive slack and no violating paths.
– The timing report marks all key setup checks as MET.